# Fitting Fourier Series

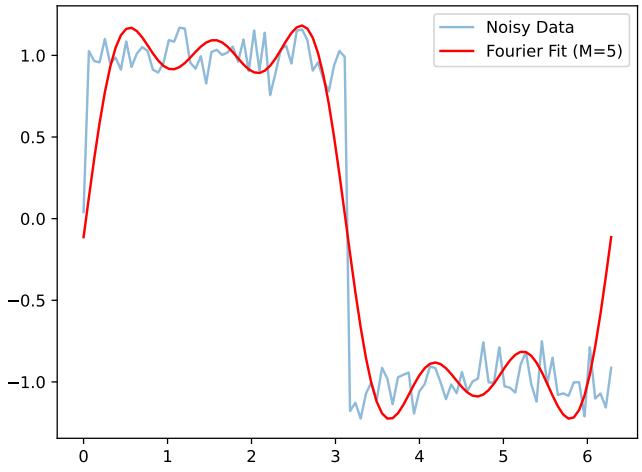
Let $f(t)$ be a periodic function with period $T$ which can be expressed as a Fourier series containing sin and cos functions with various frequencies. You are given a set of $N$ noisy data points $(t_i, y_i)$ sampled from unknown periodic process $f(t)$. Therefore, our goal is to construct an approximate of $f(t)$, namely $\hat{f}(t)$ that takes a form of $M$-term Fourier series:

$$\hat{f}(t) = a_0 + \sum_{k=1}^{M} \left( a_k \cos\left(\frac{2\pi kt}{T}\right) + b_k \sin\left(\frac{2\pi kt}{T}\right)\right).$$

The parameters are:

1. $T$ is the known period of the given signal.
2. $M$ is the number of frequency harmonics (the oder of the fit).
3. $\beta = (a_0, a_1, b_1, a_2, b_2, \ldots, a_M, b_M)$ are the coefficients we need to determine.

The problem description is to choose $\beta$ (Fourier coefficients) such that the sum of squared residuals: $\sum_{i=1}^{N} |f(t_i) - \hat{f}(t_i)|^2$ is minimized. See an example of fitting result in the figure.




# Task

For a given noisy periodic signal, and a given $M$ order of Fourier expression, find the Fourier coefficient vector $\beta$. Write a function named `fit_fourier_series(t, y, T, M)`:.

## Input Format

- $t$: a time vector of size $N$ as a 1D array.
- $y$: noisy sample values of $f(t)$ as an $N-$length vector.
- $T > 0$: the period of $f(t)$.
- $M$: a positive integer representing a number of terms in the Fourier series.

## Output Format

- $\hat{y}$: the prediction of $y$ from the fitted Fourier series. It is an $N-$length vector.
- $\hat{\beta} \in \mathbf{R}^{2M+1}$: the estimated Fourier coefficient listed as described in the problem statement.

## Sample Input

The dataset can be called from the following code. You will use `t, y_noisy` as the data points.

```
def gen_squarewave(T, N):
    np.random.seed(34)
    t = np.linspace(0, T, N)
    y_pure = np.sign(np.sin(t)) # sign is 1 or -1
    y_noisy = y_pure + 0.1 * np.random.randn(N)

    return t, y_noisy

if __name__ == "__main__":
    T = 2 * np.pi
    N = 100
    t, y_noisy = gen_squarewave(T,N)
```

## Implementation

**Goal:** Fill in the following function:

```
def fit_fourier_series(t, y, T, M):
    ...
    return yhat, beta
```

Make sure you import any library you use to solve the problem.

## Sample Output

```
import numpy as np

if __name__ == "__main__":
    import matplotlib.pyplot as plt
    M = 5  # Number of harmonics
    yhat, beta = fit_fourier_series(t, y_noisy, T, M)

    # You may try plotting to see the fitted result
    plt.plot(t, y_noisy, label='Noisy Data', alpha=0.5)
    plt.plot(t, yhat, color='red', label=f'Fourier Fit (M={M})')
    plt.legend()
    plt.show()
```